

SD6 — Certificate Management Service (the team's own domain)

JD (../0.req/oracle senior ic3.pdf): PKI team — "certificate lifecycle management, key management, trust services, cryptographic infrastructure ... secure, multi-tenant cloud deployments ... backward compatibility." This is the design most likely to come from the hiring manager or key interviewer. Cross-ref: ../4.oci-architecture.md §3 (OCI Certificates), §5 (TLS). Tags: [hot path] · [async] · [control plane]. ? = follow-up + answer.

Index

0. Time budget
 1. Why this is hard
 2. Crux
 3. Clarifying questions
 4. FRs / NFRs
 5. Back-of-envelope
 6. High-level design
 7. API
 8. Data model
 9. Deep dives (4)
 10. Hygiene + ops
 11. Pop-up questions
 12. Trade-offs + defeaters
 13. Wrap-up + card
 14. References
-

§0 Time budget

Clarify 4 · why-hard 3 · FR/NFR 4 · HLD 9 · API+schema 5 · deep dives 12 (rotation + CP/DP survival first) · wrap 3.

§1 Why this is hard

naive: openssl on a box + certs in S3 + a wiki page saying "renew yearly"
breaks at: someone forgets → expired cert → hard outage; private key on a laptop → game over

- Naive: manual issuance, keys on disk, calendar reminders
- First break: **expiry is a time bomb with 100% probability** — the canonical industry outage
- Second: key custody — any copy of a CA private key outside an HSM is a compromise story

- Minimum primitives: HSM-held keys (sign inside, never export) · versioned certs with overlapping validity · consumers that cache and hot-reload
- What the user wants: "TLS just works; rotation never wakes me; compromise is containable"
- Cost of being wrong: expired = full outage of every dependent service; leaked CA key = re-issue the world

🔑 "This is a **control-plane correctness problem with a data-plane survival constraint**: low QPS, but every mistake is either an outage-by-timer or a security incident."

\$2 Crux

- Rotation without downtime: two valid versions must overlap, and *consumers* must switch — the service can't force them
- Static stability: TLS handshakes (data plane) must keep working while this service (control plane) is down
- Key custody: issuance = an HSM signing operation, audited, never a key export
- Blast radius: two-tier CA so root compromise ≠ issuing-CA compromise; per-tenant isolation (compartments)

\$3 Clarifying questions

Question	Assumed
Internal mTLS or public?	internal-first (service identities); public via external CA later
Scale?	1M leaf certs, 10k services, cert TTL 90 d, CA TTL 5 y
Who consumes?	LBs, gateways, sidecars — pull + cache model
Revocation required?	yes — CRL primary, OCSP later
Compliance?	full audit of every issue/revoke/rotate; keys FIPS HSM

\$4 FRs / NFRs

- FR: create CA (root/subordinate) · issue leaf (CSR or managed key) · renew/rotate with versioning · revoke + publish CRL · get cert/chain/bundle · list expiring
- NFR

Requirement	Target	Enforced by
Issue latency	p99 < 2 s (HSM sign ~10–50 ms)	queue + HSM pool
Read (get cert)	p99 < 50 ms, 99.99%	read replicas + CDN-able bundle endpoint
Rotation	zero-downtime; overlap ≥ 7 d	version state machine, deprecation window
Consistency	strong on issuance/revocation state	single-writer per CA (serialize on CA row)
Durability	certs + audit immutable, 7 y	append-only audit store, WORM archive
Security	keys never leave HSM; all ops audited	Vault/HSM integration; authz per compartment

\$5 Back-of-envelope

- 1M leafs / 90 d TTL, renew at 2/3 life → ~16k renewals/day ≈ 0.2/s avg — **tiny writes**; bursts when a CA rotates (fan-out) → queue
- Reads: 10k services × cert fetch on boot/refresh (1/h) ≈ 3/s + spikes on fleet restart → cache bundle, ETag
- CRL: revoked << 1% → CRL file ~KBs–MBs; publish to object storage/CDN, re-sign every 24 h max
- So what: this is *not* a scale problem — say so, and spend the time on rotation and failure containment

\$6 High-level design

```
[control plane] Client/CI → API GW (authz, per-tenant) → Cert API svc → Postgres (CAs, certs,
versions, audit)
                                     | issue/rotate/revoke = async work request (202 + wr_id)
[async]      Issuance worker → Vault/HSM: sign(CSR|template) → store new version PENDING→CURRENT →
event
[async]      Rotation scheduler → finds certs at 2/3 lifetime → enqueue renew (idempotent per
version)
[async]      CRL builder → re-sign CRL on revoke + every 24 h → object storage → CDN
[hot path]   Consumers (LB/sidecar) – GET cert bundle (ETag; long cache) — survive CP outage on
cached cert
```

- Issuance is a work request (sd14): 202 + poll — HSM latency and CA serialization stay off the sync path
- Data plane never *calls* this service during a handshake — it consumes pushed/pulled material ahead of time

\$7 API

```
POST /v1/cas                {name, type: ROOT|SUBORDINATE, parent_ca_id?, key_algo, ttl}
POST /v1/cas/{id}:rotate    → 202 {work_request_id}
POST /v1/certs              {ca_id, csr | managed_key_spec, san[], ttl_days, auto_renew: true}
→ 202 {work_request_id}     then GET /v1/certs/{id} → CURRENT version + chain
GET /v1/certs/{id}/bundle   → PEM chain, ETag (consumers poll this)
POST /v1/certs/{id}:renew    → 202 (idempotent per current version)
POST /v1/certs/{id}:revoke   {reason} → 202; lands in next CRL
GET /v1/cas/{id}/crl        → signed CRL (also on CDN)
GET /v1/certs?expiring_before= → list (ops sweep)
```

- Versioning + backward compat (JD keyword): additive fields only; /v1 frozen; deprecation with sunset headers

\$8 Data model

```

CREATE TABLE cas (
  ca_id          uuid PRIMARY KEY,
  tenant_id      uuid NOT NULL,
  parent_ca_id   uuid REFERENCES cas,      -- NULL = root
  name           text NOT NULL,
  key_ref        text NOT NULL,           -- HSM key handle, NEVER key material
  state          text NOT NULL,           -- CREATING|ACTIVE|DISABLED|DELETED
  not_after      timestampz NOT NULL,
  version        bigint NOT NULL DEFAULT 0 -- optimistic lock; serializes issuance per CA
);

CREATE TABLE certificates (
  cert_id        uuid PRIMARY KEY,
  tenant_id      uuid NOT NULL,
  ca_id          uuid NOT NULL REFERENCES cas,
  name           text NOT NULL,
  auto_renew     boolean NOT NULL DEFAULT true,
  current_version int
);

CREATE TABLE cert_versions (
  cert_id        uuid REFERENCES certificates,
  version_num    int,
  state          text NOT NULL, -- PENDING|CURRENT|PREVIOUS|DEPRECATED|REVOKED|FAILED
  serial         numeric(49) UNIQUE NOT NULL, -- X.509 serial, per-CA unique
  pem_chain      text NOT NULL,           -- public material only
  not_before     timestampz NOT NULL,
  not_after      timestampz NOT NULL,
  revoked_at     timestampz, revocation_reason text,
  PRIMARY KEY (cert_id, version_num)
);
CREATE INDEX cv_expiring ON cert_versions (not_after) WHERE state = 'CURRENT';

CREATE TABLE audit_log (
  id             bigserial PRIMARY KEY,
  tenant_id      uuid, actor text, action text,
  resource       text, detail jsonb,
  at             timestampz NOT NULL DEFAULT now()
);

```

❓ *Where are private keys?* → "CA keys: HSM only, referenced by handle. Leaf keys: customer-held (CSR flow) or in the vault under the tenant's key — the cert service stores public material and references, never key bytes. That single sentence is the security review."

§9 Deep dives

9.1 Rotation state machine (the hard part — lead with this)

```

issue v(n+1): PENDING —HSM sign ok→ CURRENT      (v(n) → PREVIOUS)
overlap window (≥7 d): both v(n+1) CURRENT and v(n) PREVIOUS validate
deprecate: PREVIOUS —after window & adoption check→ DEPRECATED (still parseable, alarms if seen)

```

- Renew at 2/3 lifetime (90 d cert → day 60): leaves 30 d to detect + fix consumers that didn't switch
- Adoption check before deprecating: data-plane telemetry — "which cert serial did handshakes use in the last 24 h?" — don't deprecate while old serial still serves traffic
- CA rotation is the same machine one level up, plus **cross-signing**: new CA cert signed by old CA → consumers trusting old root still validate chains issued by the new one during migration

```

def tick_renewals(db, queue, now):
    """Rotation scheduler: idempotent per (cert, version) so double-enqueue is safe."""
    rows = db.query("""
        SELECT c.cert_id, cv.version_num, cv.not_before, cv.not_after
        FROM certificates c JOIN cert_versions cv
        ON cv.cert_id = c.cert_id AND cv.version_num = c.current_version
        WHERE c.auto_renew AND cv.state = 'CURRENT'
        AND %s >= cv.not_before + (cv.not_after - cv.not_before) * 2 / 3
        AND NOT EXISTS (SELECT 1 FROM cert_versions p
                        WHERE p.cert_id = c.cert_id
                        AND p.version_num = c.current_version + 1)""", (now,))

    for r in rows:
        queue.send(dedup_id=f"renew:{r.cert_id}:{r.version_num}",
                  body={"cert_id": r.cert_id, "new_version": r.version_num + 1})
# Idempotency: the NOT EXISTS + dedup_id makes re-runs no-ops.

```

```

// Issuance worker: HSM sign + version flip in one guarded transaction
@Transactional
public void issueVersion(UUID certId, int newVersion) {
    Cert cert = repo.lockCert(certId);           // SELECT ... FOR UPDATE
    if (repo.versionExists(certId, newVersion)) return; // idempotent replay
    byte[] csr = buildCsr(cert);
    byte[] pem = hsm.sign(cert.caKeyRef(), csr); // sign inside HSM, audited
    repo.insertVersion(certId, newVersion, pem, State.PENDING);
    repo.flipCurrent(certId, newVersion);        // PENDING→CURRENT, old→PREVIOUS
    audit.log("cert.rotate", certId, Map.of("version", newVersion));
    events.publish(new CertRotated(certId, newVersion)); // consumers refresh on event or poll
}

```

9.2 Static stability: data plane survives CP outage

- Consumers cache the full chain + key locally (disk), refresh on interval/event with ETag
- Handshakes never depend on a live call to this service — if it's down for a day, TLS keeps working; only *new issuance* pauses
- Corollary that interviewers probe: **cache on boot** — a consumer that fetches on startup with no local copy dies in a CP outage → ship last-known-good in the deploy artifact or persist cache to disk

- Alarm on "certs expiring < 14 d" — during a long CP outage this is the countdown clock

9.3 Revocation: CRL freshness vs availability

- Revoke = state flip + audit + CRL rebuild (signed, versioned, `nextUpdate` $\leq$ 24 h)
- Clients: CRL fetch is cached; **soft-fail** (can't fetch $\rightarrow$ accept) is the industry default — say it: "revocation is advisory in practice; short TTLs (90 d $\rightarrow$ 13 d for public web) are the real mitigation"
- OCSP + stapling as the follow-up: server staples a signed freshness proof; solves client fetch, adds responder availability as a new dependency

9.4 Multi-tenancy + authz (JD keyword)

- Every resource in a tenant compartment; policy checks at gateway (`allow group X to manage certificates in compartment Y` — OCI-style)
- Quotas: certs per tenant, issue-rate per tenant (token bucket, `sd01`)
- Cross-tenant isolation test: tenant A's token must 404 (not 403) on B's cert ids — no existence leak

§10 Hygiene + ops

- Metrics: issuance success rate, HSM latency, **expiring-in-N-days count** (page at 14 d), CRL age, rotation adoption lag, audit write failures (page immediately)
- Runbook: expired-cert incident $\rightarrow$ issue emergency short-TTL cert from subordinate CA $\rightarrow$ hot-reload consumer $\rightarrow$ RCA why adoption alarm missed it
- Deploy: canary; API is versioned; DB migrations expand $\rightarrow$ migrate $\rightarrow$ contract (JD: backward compatibility)
- Chaos: block consumers from CP for 24 h — assert zero handshake failures (static stability test)

§11 Pop-up questions

- *Root CA compromised?* $\rightarrow$ "Two-tier means re-key subordinates from a new root, cross-sign for migration, revoke old root via trust-store updates (out-of-band). This is why the root is offline/HSM and used only to sign subordinates."
- *Why not Let's Encrypt everything?* $\rightarrow$ "Public trust for public endpoints, sure. Internal mTLS wants private CAs: no rate limits, internal names, custom policies, and no dependency on an external issuer for service identity."
- *Cert per pod at scale?* $\rightarrow$ "Short-TTL (24 h) certs, no revocation reliance, SPIFFE-style identity; issuance QPS rises 100 $\times$ — now it is a scale problem: HSM pool, per-CA sharding, or an intermediate signing tier with software keys for leafs."
- *How do consumers hot-reload?* $\rightarrow$ "Watch file/event $\rightarrow$ rebuild SSLContext; Envoy SDS or Java `KeyManager` reload. Never require process restart — restart storms are self-inflicted outages."
- *Serial uniqueness?* $\rightarrow$ "Per-CA unique, ≥ 64 bits random (CA/Browser rules) — DB unique constraint + random generation, not a counter (predictable serials aided past attacks)."

§12 Trade-offs + defeaters

Decision	Chosen	Alternative	Flips when
Hierarchy	two-tier	single root	dev/test only

Decision	Chosen	Alternative	Flips when
Renewal	auto at 2/3 life, versioned	manual	never (manual = the outage generator)
Revocation	CRL primary	OCSP(+stapling)	browser-facing public certs
Issuance	async work request	sync	interactive dev flows (add a fast lane)
Consistency	strong, single region per CA	multi-region active-active	only via per-region subordinate CAs

- Defeaters → "Store the CA key encrypted in the DB" — encrypted-at-rest is not custody; sign inside the HSM, key never exists in service memory → "Rotation = swap the cert atomically" — consumers cache; without overlap you break every cached client at once → "Rely on revocation for compromise" — soft-fail clients; short TTL is the honest control

§13 Wrap-up + card

- Pitch: control-plane cert lifecycle over HSM-held keys; versioned rotation with overlap + adoption checks; CRL to CDN; data plane statically stable on cached bundles
- Why hard: expiry-by-timer + key custody + you can't force consumers to switch
- 5 numbers: 90 d leaf TTL, renew at day 60 · overlap ≥ 7 d · CRL nextUpdate 24 h · issue p99 2 s · page at 14 d to expiry
- 3 failures: consumer never adopts (adoption telemetry) · CP outage (static stability) · root compromise (two-tier + cross-sign)
- 3 defeaters: encrypted-DB key custody · atomic swap rotation · revocation-as-primary-control
- Done when: rotation state machine drawn from memory; static-stability answer in 30 s; schema written cold

§14 References

- OCI Certificates docs (overview, CA hierarchy, rotation, BYOCA) — [../4.oci-architecture.md](#) §9 links
- AWS Private CA + ACM rotation docs (analogy source)
- SPIFFE/SPIRE — workload identity model for the "cert per pod" follow-up
- Smallstep blog — internal PKI patterns; CA/Browser Forum baseline requirements (serials, TTLs)